SHRSS AEMaaCS Indexing & Performance Guide

SHRSS AEMaaCS Indexing & Performance Guide

Indexes, Queries, and Performance for Architects & Developers

---

1. Purpose & Scope

This volume is a **deep-dive reference** for SHRSS developers and architects on:

- How indexing works in **AEM as a Cloud Service** (AEMaaCS)
- How to design and maintain **custom index definitions** for SHRSS (Jobs, Events, News, Locations, etc.)
- How to **troubleshoot and tune** query & indexing performance using AEM tools (Query Performance/Analyzer, Query Builder Debugger, logs)
- How to avoid **Cloud Service–specific anti-patterns** that break pipelines or degrade performance

It goes **deeper than Experience League** but continuously points back to the relevant official docs.

Key Experience League references (read alongside this doc)

- **Indexing concepts & lifecycle (AEMaaCS)**
  - Content Search and Indexing
  - Search and indexing in AEM as a Cloud Service
- **Query & indexing patterns, best practices**
  - Query and Indexing Best Practices
  - Indexing best practices in AEM
- **Tools & troubleshooting**
  - How to investigate search related issues in AEM
  - Adobe Experience Manager: Handle “Query Without Index Detected” Alert
  - Custom index reindexing issues in AEM as a Cloud Service - Sites
- **Oak internals (for when you need to go really deep)**
  - Lucene Index

- Query Engine

---

2. Indexing in AEMaaCS: Mental Model

2.1 Core principles

In AEMaaCS:

- **All supported custom indexes are Oak Lucene indexes**
  No property indexes, no Solr indexes. Lucene-only, `compatVersion=2` is mandatory.
  See Content Search and Indexing.

- **Index definitions are code**
  - Stored as `oak:QueryIndexDefinition` nodes under `/oak:index` at runtime.
  - In the project: under `ui.apps/src/main/content/jcr_root/_oak_index`.
  - Deployed via **Cloud Manager pipeline**, not via CRXDE on cloud.

- **Blue–green deployment of indexes**
  - Each product-provided index has a version in the name: e.g. `/oak:index/damAssetLucene-11`.
  - Customizations are **separate nodes**: `damAssetLucene-11-custom-1`, etc.
  - Indexing runs as part of the deployment; the new version takes traffic only after indexing completes.

- **Cost-based query engine**
  - Every index estimates its cost for a query.
  - The engine picks the lowest-cost index.
  - If no suitable index exists, the query **falls back to traversal** (slow, and in AEMaaCS often triggers alerts).

2.2 Index types relevant to SHRSS

Typical OOTB indexes you depend on:

Index	Purpose / Node Type	SHRSS Relevance
`cqPageLucene-*`	Fulltext + property for `cq:Page`	Site search, page authoring lookups
`damAssetLucene-*`	Assets `dam:Asset`	DAM search, CF binaries, image search

Index	Purpose / Node Type	SHRSS Relevance
`fragments` / CF-related	Content Fragments / GraphQL	Headless / CF-based APIs
`pathreference`	Reference search by path	Link checking, reference lookups

For SHRSS Phase 1, the **heavy hitters** are:

- **Sites / pages**: lists & search for **Jobs, Events, News, Locations** via components (often QueryBuilder or JCR-SQL2).
- **Content Fragments**: behind Jobs/Events/Locations data, and **GraphQL persisted queries**.
- **Assets**: images, PDFs, and potentially asset-search facets.

You should rarely create brand-new indexes; in most cases you **extend an existing OOTB index**.

---

3. SHRSS Use Cases & How Indexing Supports Them

This section is about recognizing the **query patterns** in SHRSS and how they should drive index design.

3.1 Jobs listings

Typical patterns:

- Filter by **location**, **brand/property**, **job category**, **employment type**, **status**.
- Sort by **posting date** or **job title**.
- Maybe full-text search on job title/description.

Indexing implications:

- The properties you filter/sort on must be **indexed as properties**:
  - e.g. `jcr:content/data/<fieldName>` as `propertyIndex=true`, `ordered=true` for sort.
- If Jobs are backed by **CFs**, ensure the CF-relevant index (often a fragments index or `damAssetLucene` depending on implementation) has those properties.

3.2 Events listings

Patterns:

- Filter by **date range**, **venue/location**, **event type**, **age restriction**.
- Sort by **start date**.

Indexing implications:

- Date fields (start/end) should have **propertyIndex + ordered** for efficient range queries and sorting.
- For venue references (e.g. path or ID), also index that property.

3.3 News listings

Patterns:

- Filter by **category**, **tags**, **publish date**, and maybe **brand**.
- Full-text search across **headline and summary**.

Indexing implications:

- Tag/category properties: property index & optionally full-text.
- Date: `ordered=true` for sort.
- Titles/summaries: usually full-text via `nodeScopeIndex=true`.

3.4 Locations / Venues

Patterns:

- Filter by **country/state/city**, **property type**, **brand**.
- Might involve **geography facets** (country, region) and map displays.

Indexing implications:

- Location attributes should be property-indexed.
- If you use facets (e.g. filter panel counts), those properties must have `facet=true` in the index definition.

The concrete index work is: **find the queries** behind these features and build/extend indexes specifically for them.

---

4. Working with OOTB Indexes (Especially `damAssetLucene`)

4.1 General guidance (Cloud Service)

Per Content Search and Indexing and Indexing best practices in AEM:

- **Prefer customizing OOTB indexes** over creating new ones for the same node type.
  - E.g. don't create `/oak:index/myDamAssets`; instead, create `damAssetLucene-<version>-custom-1`.
- **Never modify OOTB nodes in place** (`damAssetLucene-11`); always copy to `-custom-*`.

4.2 Customizing damAssetLucene-* for SHRSS

Use cases where SHRSS might need to extend damAssetLucene:

- Assets metadata used for filters (e.g. cq:tags, custom metadata like brand/property).
- Content Fragments stored as dam:Asset (for some models).
- Integration-specific flags (e.g. translation, rights).

Process (per Content Search and Indexing – Index Names & Customization):

1. **Export the current product index** from a Cloud Service environment via Package Manager.
2. Create a folder in ui.apps:

```
ui.apps/
  src/main/content/jcr_root/_oak_index/
    damAssetLucene-11-custom-1/
      .content.xml
```

3. Adjust **only** what you need:
   - Add new properties under /indexRules/dam:Asset/properties.
   - Ensure type="lucene", compatVersion="{Long}2", and async="[async,nrt]".
4. Keep all product-required nodes (e.g. tika), even if they don't exist in the SDK—Cloud docs call this out explicitly.

Key Cloud Manager rule (see Custom Code Quality Rules and Custom Code Quality Rules):

- Custom full-text index of type damAssetLucene **must be prefixed** damAssetLucene.
  Example: damAssetLucene-11-custom-1 ✅, shrssDamAssets-1-custom-1 ❌.

4.3 cqPageLucene-* customization

You might extend cqPageLucene-* when:

- Page properties are used for filters (page type, brand, property, market).
- You want facets or sort by specific page properties.

Pattern:

- Copy /oak:index/cqPageLucene-<n> → cqPageLucene-<n>-custom-1.
- Add your page properties under /indexRules/cq:Page/properties with:
  - propertyIndex="{Boolean}true" for equality filters.

- `ordered="{Boolean}true"` for sorts.

---

5. Designing Custom Indexes from Queries (SHRSS-Style)

5.1 Start with the query (never with the index)

Whether you're using:

- **QueryBuilder** (`/libs/cq/search/content/querydebug.html`)
- **JCR-SQL2** / XPath
- **GraphQL** (for CFs)

The workflow is:

1. Write the **simplest correct query** for your use case.
2. Run it on local SDK and in **Developer Console → Query Performance** with Explain/Analyze.
3. Observe:
   - Which index is used (if any), and
   - Whether the query is **read-optimized or traversal-prone**.

Only then decide whether you need to:

- Change the **query** (preferred), or
- Extend an existing index.

5.2 Example: Jobs listing (SQL2)

Pseudo-example, adapt paths and property names to match your CF / page model.

```
SELECT  [jcr:path], [jcr:score]
FROM    [cq:Page] AS page
WHERE   ISDESCENDANTNODE(page, '/content/shrss')
  AND   page.[shr:pageType] = 'job'
  AND   page.[shr:brand] = 'Hard Rock'
  AND   page.[shr:locationRegion] = 'Florida'
ORDER BY page.[shr:postedDate] DESC
```

Explain results may show:

- **Plan**: `lucene:cqPageLucene-2(/oak:index/cqPageLucene-2)` — good.
- **But** Read Optimization poor because `shr:brand` & `shr:locationRegion` are not indexed.

Remedy:

- Customize `cqPageLucene-*` to add these properties in `indexRules/cq:Page/properties`:

```
<shrBrand
    jcr:primaryType="nt:unstructured"
    name="shr:brand"
    propertyIndex="{Boolean}true"
    />
<shrLocationRegion
    jcr:primaryType="nt:unstructured"
    name="shr:locationRegion"
    propertyIndex="{Boolean}true"
    />
<shrPostedDate
    jcr:primaryType="nt:unstructured"
    name="shr:postedDate"
    propertyIndex="{Boolean}true"
    ordered="{Boolean}true"
    type="Date"/>
```

5.3 Example: Events CF listing (GraphQL)

GraphQL queries ultimately translate to underlying index usage (often a CF index and/or `damAssetLucene`). Per Query and Indexing Best Practices:

- Ensure CF model fields used for filtering (date, venue, category) are indexed.
- Sometimes you add properties to customized `damAssetLucene` so CF search avoids generic lucene.

Pattern:

- Identify the underlying node type & path of the CFs.
- Extend the relevant index (CF index or `damAssetLucene-*`) so those properties are property-indexed.

---

6. Index Definition Anatomy (Lucene)

This section is more "Oak-level", but understanding it gives you fine-grained control.

6.1 Top-level properties

From Lucene Index and Content Search and Indexing:

```
<jcr:root
    jcr:primaryType="oak:QueryIndexDefinition"
    type="lucene"
    compatVersion="{Long}2"
    async="[async,nrt]"
    evaluatePathRestrictions="{Boolean}true"
    includedPaths="[/content/shrss]"
    queryPaths="[/content/shrss]"
    tags="['shrssJobs']"
    >
    <indexRules jcr:primaryType="nt:unstructured">
        ...
    </indexRules>
</jcr:root>
```

Key attributes you'll tune most often:

Property	What it does	Notes / Gotchas
`type`	Must be `"lucene"` in AEMaaCS.	Anything else fails Cloud Manager.
`compatVersion`	Must be 2.	AEMaaCS requirement.
`async`	Usually `"[async,nrt]"` for fulltext; `"[async]"` also valid.	Wrong values can break reindexing (Custom index reindexing issues).
`includedPaths`	Subtree(s) to index.	Always set this; avoid global `/`.
`queryPaths`	Path scope that must match query path restriction.	For most cases, set equal to `includedPaths`.
`evaluatePathRestrictions`	Use index to apply `ISDESCENDANTNODE/ISCHILDNODE`.	Essential when you restrict by path.
`tags`	Index tags used with `option(index tag <tag>)` in queries.	Don't reuse product tags; use custom tags.

Avoid using `excludedPaths` unless you really need it; per Indexing best practices in AEM it's better to narrow via `includedPaths` + `queryPaths`.

6.2 `indexRules` and `properties`

Under `indexRules`, define per-node-type rules:

```
<indexRules jcr:primaryType="nt:unstructured">
  <cq:Page jcr:primaryType="nt:unstructured">
    <properties jcr:primaryType="nt:unstructured">
      <shrPostedDate
          jcr:primaryType="nt:unstructured"
          name="shr:postedDate"
          propertyIndex="{Boolean}true"
          ordered="{Boolean}true"
          type="Date"/>
      <shrCategory
          jcr:primaryType="nt:unstructured"
          name="shr:category"
          propertyIndex="{Boolean}true"
          />
    </properties>
  </cq:Page>
</indexRules>
```

Common property-level flags:

Property	Meaning	When to use
`propertyIndex`	Enables equality / range queries on this property.	Any filter field (date, category, location, status).
`nodeScopeIndex`	Include property in full-text `jcr:contains(., 'x')`.	For fields you want to match in global search.
`analyzed`	Tokenize / normalize value for full-text.	For text fields with word-level search.
`ordered`	Enables efficient sorting on this property.	For date & title sorts.
`useInExcerpt`	Store value in index for excerpt feature.	Only if you actually use excerpts.

Property	Meaning	When to use
`facet`	Enable facet counts on this property.	For filter counts (e.g. by location or category).
`type`	Specifies property type (`String`, `Date` etc.).	Important for correct ordering & comparisons.

---

7. Best Practices & Anti-Patterns (Cloud Service–Specific)

7.1 Where and how to store index definitions

Per Indexing best practices in AEM and Custom Code Quality Rules:

- Place custom index definitions under:
  - `ui.apps/src/main/content/jcr_root/_oak_index/`
- Ensure package `properties.xml` has:

```
<property name="allowIndexDefinitions" value="true"/>
<property name="noIntermediateSaves" value="true"/>
```

- **Do not** put index definitions in `ui.content` or random content modules (Cloud Manager will flag).

7.2 Naming conventions

From Content Search and Indexing – Index Names:

- OOTB index:
  `/oak:index/damAssetLucene-11`
- Customization of OOTB index:
  `/oak:index/damAssetLucene-11-custom-1`
- Fully custom index:
  `/oak:index/shrss.jobs-1-custom-1`
  (use a project prefix like `shrss.`)

Rules:

- Always end with `-custom-<n>`.
- For custom full-text indexes of type `damAssetLucene`, name **must start** with `damAssetLucene`.

7.3 Forbidden / discouraged patterns

Common anti-patterns called out in docs and Cloud Manager rules:

- **Custom index with `type != lucene`** → fails in AEMaaCS.
  See Custom Code Quality Rules – IndexType.
- **Setting `reindex` or `seed` properties** on definitions → prohibited.
  Reindexing is automatic in cloud.
- **Custom `dam:Asset` full-text index** instead of customizing `damAssetLucene-*` → leads to conflicts and poor performance.
  Use a `damAssetLucene-*` customization instead.
- **Generic "catch-all" lucene indexes** (like old `/oak:index/lucene-*`) → deprecated and removed.
  See Generic Lucene Index Removal.
- **Over-indexing everything**:
  - Index every property "just in case".
  - Large `includedPaths` like `/content` for all sites.
    This bloats index size and degrades performance.
- **Queries executed from components on every request**
  Especially expensive for pages with many list components. Per Query and Indexing Best Practices, consider **precomputing** or caching.

---

8. Performance & Query Design

8.1 How cost and traversal work (pragmatic view)

From Query Engine and Query and Indexing Best Practices:

- Every query has:
  - A **filter** (what you're searching for).
  - A **path restriction** (`ISDESCENDANTNODE`, etc.).
- Each index estimates how many nodes it will need to read; that's the "cost".
- If no index can handle the filter efficiently, Oak uses the **traversal index**:
  - Walks the subtree node-by-node up to a limit (~100k nodes by default).
  - AEMaaCS may trigger a **"Query Without Index Detected"** alert.

See Adobe Experience Manager: Handle "Query Without Index Detected" Alert.

8.2 SHRSS query patterns to watch

The riskiest patterns in SHRSS:

- "All Jobs in all properties, filter later in code":
  - Query missing path restriction or with `ISDESCENDANTNODE('/', ...)`.
- Big OR chains:

  ```
  WHERE category = 'A' OR category = 'B' OR category = 'C'
  ```

  Sometimes better rewritten as `IN` or simplified filters.
- Full-text queries on unindexed properties:

  ```
  WHERE CONTAINS([jcr:content/metadata/myCustomField], 'value')
  ```

  when `myCustomField` is neither fulltext nor property-indexed.

Strategies:

- Always include a **path restriction** matching `includedPaths`/`queryPaths`.
- Ensure every filter field used in your query has a **propertyIndex** entry.
- For multi-facet pages (Jobs or Events with many filters), consider:
  - Precomputing some aggregates.
  - Splitting complex queries into smaller ones and joining results in code if needed.

---

9. Tools & Troubleshooting: Query Analyzer, Performance Console, Debuggers

9.1 Developer Console – Query Performance Tool (Cloud Service)

Per Query and Indexing Best Practices and Search and indexing in AEM as a Cloud Service:

Where:
Cloud Manager → your environment → Developer Console → *Query* → **Query Performance**

What you get:

- List of **slow/top queries** executed in the environment.

- For each query:
  - Query text (SQL2/XPath).
  - Execution time, node read count.
  - **Query plan** (index used, cost, traversal info).

How to use it (Query Analyzer workflow):

1. **Identify slow query** from the Query Performance list.
2. Copy the query into the **Explain Query / Analyze** tab.
3. Run **Explain** (without full execution) to see:
   - Which index is chosen (`lucene:cqPageLucene-2`, `lucene:damAssetLucene-11`, etc.).
   - Estimated node reads and cost.
4. Run **Analyze** or full execution for deeper metrics (e.g. read optimization).

If you see:

- `traversal` or generic lucene usage (`/oak:index/lucene-*`) → need new or updated index.
- Low read optimization, high node read count → refine query or index properties.

9.2 Query Builder Debugger (classic, still useful)

Per How to investigate search related issues in AEM:

- Accessible at: `/libs/cq/search/content/querydebug.html` (local/SDK, and some cloud envs).
- Good for:
  - Exploring QueryBuilder queries for SHRSS components.
  - Seeing the **translated SQL2** and **query plan**.
  - Iteratively tuning predicates and filters.

Recommended workflow:

1. Copy the predicate map from your component's implementation.
2. Paste into Query Debugger, run, and check **Execution Time** and **Plan**.
3. If the plan doesn't show a suitable index, adjust:
   - Query filters (add `path`, remove unneeded predicates).
   - Or extend the index backing the query.

9.3 Logs & Alerts

Key symptoms & where to look:

- **"Query Without Index Detected" email alert**
  See Adobe Experience Manager: Handle "Query Without Index Detected" Alert.

Action:

- Identify the offending query from logs.
- Run it through Query Performance / Explain Query.
- Either optimize query or add index.

- **Traversal warnings** in logs:
  - Look for messages mentioning TRAVERSAL and query text.
  - Same remediation.

- **Index-related Cloud Manager build failures**:
  - Violations of index type (non-lucene), `compatVersion`, invalid properties (`reindex`, `seed`), or invalid damAssetLucene naming.
  - Fix definitions as per Custom Code Quality Rules.

---

10. Operational Playbook for Index Changes (SHRSS)

This is the **workflow you should standardize** for SHRSS development.

10.1 Design phase

1. For a new feature (Jobs filter, Events facet, etc.), write down:
   - The **query** the feature needs (SQL2, QueryBuilder, GraphQL).
   - Fields to **filter** and **sort** on.
2. Check existing indexes via:
   - Developer Console (download index stats).
   - List of `/oak:index/*` from local SDK.
3. Decide:
   - Can you **reuse OOTB**?
   - Do you need to **customize** an index (`cqPageLucene-*`, `damAssetLucene-*`)?
   - Or do you need a **new index** (for completely custom node types)?

10.2 Implementation phase

1. Implement the **index definition** under `ui.apps/_oak_index` with proper naming.
2. Add only **required properties** under `indexRules`.

3. Ensure:
    - `type="lucene"`, `compatVersion=2`, `async` set correctly.
    - `includedPaths` and `queryPaths` are as narrow as is reasonable.
4. Run **local SDK**:
    - Deploy index via `mvn clean install -PautoInstallSinglePackage`.
    - Run your query via:
        - Query Builder Debugger, or
        - `/libs/granite/operations/content/diagnosistools/queryPerformance.html` on SDK.

10.3 Validation in Cloud

1. Deploy to **DEV** via non-prod pipeline.
2. Use Developer Console:
    - Confirm index is present in stats.
    - Use Query Performance/Explain to validate:
        - Right index is used.
        - Read optimization is good (no traversal).
3. Load test the feature page(s) (Jobs / Events / News).

10.4 Rollout & Maintenance

- Include index changes in **change & release documentation**.
- When Adobe updates OOTB indexes (e.g. `damAssetLucene-11` → `damAssetLucene-12`):
    - Follow Content Search and Indexing – Merging Customizations to update your custom versions.
- Periodically review:
    - Query Performance reports for slow queries.
    - Index stats for large or underused indexes.

---

11. SHRSS-Specific Recommendations / "House Rules"

These are good SHRSS norms to adopt on top of generic best practices.

1. **No new full-text indexes on `dam:Asset`**
    - Always customize `damAssetLucene-*` instead.
2. **Every list component has a documented query & index**

- For Jobs, Events, News, Locations lists:
  - Document the query pattern.
  - Link to the index definition that supports it.

3. **No “wildcard” indexes for convenience**
   - Avoid `name=".*"` universal property patterns unless you have a focused use case.
4. **Path restrictions required for all repository queries**
   - Any repository query in SHRSS code must:
     - Include a path restriction under `/content/shrss`, `/content/dam/shrss`, etc.
     - Or explicitly justify why it doesn’t (rare).
5. **Use tags for complex custom indexes**
   - For complex SHRSS-specific search use cases, tag your index and use `option(index tag shrssJobs)` to ensure the correct index is used.
6. **Treat indexing changes as breaking changes**
   - Never change index definitions in prod without going through:
     - DEV → STAGE → PROD pipeline, with checks at each step.

---

12. Quick Reference

12.1 “What do I check first?” for a slow Jobs/Events/News list

1. Reproduce on DEV.
2. Capture underlying query (QueryBuilder, SQL2, GraphQL).
3. Run through:
   - Local SDK Query Debugger (if applicable).
   - Developer Console → Query Performance → Explain Query.
4. Confirm:
   - Index used? Which one?
   - Read optimization adequate?
5. If not:
   - Add/extend index properties or adjust query.
6. Re-deploy, re-test, re-measure.

12.2 When to create a new custom index

- Node type is **custom** and not covered by OOTB indexes.
- Queries cannot be efficiently expressed via existing indexes (even with customization).
- Scope is **narrow** and well-understood (e.g. SHRSS-specific app subtree).

Name pattern: `/oak:index/shrss.<logical-name>-1-custom-1`.

References

- Content Search and Indexing
- Search and indexing in AEM as a Cloud Service
- Query and Indexing Best Practices
- Indexing best practices in AEM
- Adobe Experience Manager: Handle “Query Without Index Detected” Alert
- Custom index reindexing issues in AEM as a Cloud Service - Sites
- Generic Lucene Index Removal
- Custom Code Quality Rules
- Lucene Index
- Query Engine
- How to investigate search related issues in AEM
- Cloud 5 AEM Search and Indexing